

2. Implementation Details

Our model represents trait interdependency networks using directed acyclic graphs (DAGs) $G = (V, E)$, where traits correspond to nodes V and prerequisite relationships to directed edges E . For any edge $(i, j) \in E$, trait i is a parent of trait j , and j is a child of i . The ancestor set of trait j includes all traits reachable by following edges backward, while descendants are reachable following edges forward. Each graph contains a root node (trait 0) representing the naive state with no learned traits. The parameter R measures prerequisite density as the average number of ancestors per trait excluding the root. This quantifies the overall constraint level within the dependency structure.

2.1 Graph generation

2.1.1 Generation of DAGs. We systematically generate all possible interdependency structures (graphs) with the number of traits n ranging from 3 to 8. All generated graphs must be DAGs. This constraint ensures that the dependency structures do not contain cycles; circular dependencies among traits would render some traits unlearnable.

To ensure our analysis covers only unique dependency structures, we apply two filtering steps to all generated graphs. First, we perform a transitivity reduction on all graphs to eliminate redundant edges. In dependency structures, redundancies can occur when indirect relationships are also represented directly. For example, in a graph with three traits and deterministic dependencies (i.e., binary prerequisites where traits are either required or not required), if trait C depends on trait B, and trait B depends on trait A, then a direct connection between trait A and trait C is redundant as trait C already implicitly depends on trait A through trait B. Removing these redundant connections ensures that the graph only contains the minimal set of edges needed to represent all learning constraints.

After generating the graphs, we remove all isomorphic graphs using the 'nauty' algorithm (McKay and Piperno, 2014). Two graphs are isomorphic if there exists a renaming of the nodes that preserves the adjacency relations. Eliminating isomorphic duplicates ensures that each dependency structure in our analysis is unique in the constraints it enforces. Figure S1 shows a representative subset of the generated graphs.

2.1.2 Graph Sampling. For graphs with more than 8 traits (cf. Fig. 2B,C of the main text) computational demands make an exhaustive analysis of all possible graphs impractical. We therefore sampled graphs according to the following procedure, covering a wide range of constraints (R). For a given number of traits n , we generated the minimally and maximally constrained graph ($R = 0$ and $R = \frac{n}{2} - 1$, respectively). We then divided the range between the minimum and maximum R into 50 evenly spaced bins of R and generated a graph in each bin.

2.2 Trait Payoffs

Our work focuses on interdependencies between traits. We therefore keep the payoffs of traits as simple as possible. We assume that payoffs vary between traits. The distribution of payoffs across the networks can have impact on the efficiency of social learning strategies that use payoffs (e.g., when very advanced traits – far from the root – have high payoffs, novices will try to learn those traits and fail). In our analyses, we therefore consider all combinations of payoffs across traits in each network.

We assign payoffs to traits using the formula:

Commented [GU1]: is this needed?

Commented [nj2R1]: We make use of these terms later on and there were comments that these terms were not explained

Commented [GU3]: towards the root?

Commented [GU4R3]: the reader cannot orient yet, or?

Commented [nj5R3]: The edges themselves are directed, the root doesn't really matter in a way

Commented [GU6]: should we write a brief overview paragraph here, and refer to the code repository so the reader doesn't get lost?

Commented [nj7R6]: Depends on how we want to integrate this with the existing method section.

Commented [nj8R6]: Different sections here will become different branches in a github repo. I will add links to the branches at the start of each section

Commented [GU9]: i think we can immediately say "DAG" as it was in the previous para

Commented [GU10]: can you provide a reason up front why you want to do this?

Commented [GU11R10]: so move this up: Removing these redundant connections ensures that the graph only contains the minimal set of edges needed to represent all learning constraints.

Commented [GU12]: let's make sure the reader knows what this means

Commented [NJ13R12]: Added an explanation

Commented [GU14]: i would refer to a figure with a sample of graphs here

Commented [LM15R14]: Should we also refer to these graphs in the main tekst (e.g., in the caption of fig 1)?

$$payoff_i = \frac{2(i+1)}{n+1} \text{ for } i \in \{1, 2, \dots, n\}$$

This formula generates values that are equally spaced and have a mean payoff of exactly 1. This approach creates variation for payoff bias to have an advantage over other strategies by focusing their learning on high-payoff traits (and ignoring low-payoff traits).

The arrangement of payoffs in the graph substantially affects the performance of all learning strategies, and the performance of payoff bias in particular. To control for interactions between trait payoffs and dependency structure, we evaluate all possible permutations of payoff values across traits. For each graph, we run the model for all $(n-1)!$ possible arrangements of payoffs (excluding the root trait, which always has a payoff of 0). Results shown in the figures in the main text and in the SI are then averaged across all permutations. This ensures that payoff values are distributed evenly across all trait positions in the dependency structure, creating a balanced distribution of payoffs that is independent of structural constraints.

For networks with more than 8 traits (cf. Fig. 2B,C), the computational cost of evaluating all $(n-1)!$ permutations becomes prohibitive. For analysis of these larger networks, we limit the number of permutations to 5,000 for each network, ensuring that all are unique and randomly sampled from the permutation space.

2.3 Social Learning Strategies

2.3.1 Overview. Individuals randomly sample others (“demonstrators”) in the population, so that the frequencies of traits that individuals observe is equal to their frequencies in the population. To select a trait for learning, individuals only consider traits that are not currently in their own repertoire. For this subset of observed traits, they weigh each trait according to their social learning strategy.

To obtain the probability that an individual selects trait i for learning, we multiply the frequency of trait i with the bias from the individual’s social learning strategy. For prestige-based and proximal social learning, learners consider demonstrator repertoires. To map the weighting onto traits, we calculate the frequencies f of traits as sum of the frequencies of all demonstrator repertoires which contain that trait (see below for details).

2.3.2 General Form. The general form of trait weight is:

$$W_i = f_i \cdot bias_i$$

The probability P of attempting to learn trait i is linearly proportional to its weight:

$$P_i = \frac{W_i}{\sum_k W_k}$$

2.3.3 Strength of Bias. For each social learning strategies, we use a parameter b to indicate the strength of the bias individuals use in weighting the traits they observe. Each social learning strategy uses a different feature of the trait or the demonstrator expressing the trait for weighting. When $b = 0$, social learning is random for each strategy. Results in the main text are presented with $b = 2$ for all strategies. We show results for other values for b in Fig. S4.

Commented [GU16]: do we need to say this? not sure what it means

Commented [nj17R16]: Better now?

Commented [LM18]: Not so nice to have W for trait weight and w for edge weight

Commented [GU19]: i seem unable to edit the formula but there's a typo in "frequency"

Commented [GU20]: is it my browser or is the font changing here? looks messy

Commented [nj21R20]: Fixed.

2.3.4 Random. The random strategy is not biased towards any feature of the trait, only influenced by its frequency in the population: $W_i = f_i \cdot 1$, where P_i is the probability that the learner attempts to learn trait i and f_i is the frequency of trait i in the population.

2.3.5 Payoff. The payoff strategy is biased towards traits with higher payoffs: $W_i = f_i \cdot p_i^b$, where p_i is the payoff of trait i .

2.3.6 Conformity. The conformity strategy is disproportionately biased towards more common traits: $W_i = f_i^b$.

2.3.7 Prestige. The prestige strategy is biased towards traits that are expressed by demonstrators with higher total payoffs: $W_i = \sum_{j \in D_i} f_j \cdot (\text{payoff}_j)^b$, where D_i is the set of unique demonstrator repertoires that contain trait i , f_j is the frequency of demonstrator repertoire j in the population, payoff_j is the total payoff of demonstrator repertoire j .

We also test an alternative implementation of prestige which is biased towards traits that are expressed by demonstrators with large repertoires: $W_i = \sum_{j \in D_i} f_j \cdot (|\text{repertoire}_j|)^b$, where $|\text{repertoire}_j|$ is the number of traits in demonstrator repertoire j (See Fig. S5).

2.3.8 Proximal. The proximal social learning strategy is biased towards traits expressed by demonstrators who are slightly more knowledgeable than the learner: $W_i = \sum_{j \in D_i} f_j \cdot \Delta_j^{-b}$, where Δ_j is the number of traits in the demonstrator's repertoire that are not in the learner's repertoire. Learners identify demonstrators with slightly larger skill sets and randomly select from the traits those demonstrators possess that the learner does not yet know.

2.4 Soft dependencies

In our baseline model, we assume deterministic dependencies between traits, such that traits can only be learned if an individual has already learned all prerequisites for that trait. For the analyses with non-deterministic (softer) dependencies, we considered graphs with weighted edges. Weights varied between 0 and 1, representing the probability that having learned the parent node is necessary to learn the child node. Weights of 1 mean deterministic dependencies (strong constraints on learning), and weights of 0 indicate a completely unconstrained scenario. For example, an edge from A to B with weight 0.3 indicates a 30% probability that A is required when attempting to learn B. We implement a uniform weighting approach, assigning identical weights to all edges and varying this value from 0 to 1 (Fig. 2G-I).

Furthermore, the transitivity reduction applied to unweighted graphs (section 2.1.1) cannot be directly extended to weighted graphs due to the probabilistic nature of dependencies. For example, if trait C depends on trait B with probability 0.7, and trait B depends on trait A with probability 0.5, the implicit dependency of trait C on trait A cannot be determined by simple transitivity. Unlike in unweighted graphs, knowing trait B does not guarantee knowledge of trait A.

Consequently, in weighted graphs, dependency relationships between a target node and all its ancestor nodes must be explicitly defined to accurately capture the complete dependency structure. This exponentially increases the space of possible graphs. To manage this complexity, we restrict our analysis to two boundary cases: the transitive reduction (minimal edge set) and the transitive closure (maximal edge set; shown in Fig. 2G-I) of each graph. In the transitive reduction, lacking trait A affects learning trait C only through its effect on trait B. In the transitive closure, we add a direct A→C edge, so lacking trait A directly reduces the

Commented [nj22]: Add citation for prestige as total accumulated success

Commented [LM23]: Would it make sense to move this down to the section about Learnability? In a way this is not the baseline model but a variant of it

Commented [NJ24R23]: Is it better here?

Commented [LM25]: Should we add 1 sentence on how this generalizes to >1 parent trait?

Commented [NJ26R25]: The learnability formula in the markov chain section is the best explanation for this. Maybe pull it into its own section before soft dependencies?

Commented [GU27]: i think we could repeat that we did this for 4 specific values of R, and list the number of unique networks for each

Commented [GU28R27]: just make things explicit after Fig. 2 of the main text is updated

Commented [LM29R27]: So i think we decided we choose 3 levels of these weights, and do the "normal" graph

Commented [LM30R27]: So if we would choose the same set of graphs as in the other analyses, can we remove this and the next paragraph?

Commented [GU31]: slightly easier to name them ABC?

Commented [GU32]: i thought that the set of payoffs for each graph separately is quasi uniform, not the result of the averaging process?

Commented [nj33R32]: Yeah quasi-uniform is not the right word. Maybe something like uniform in expectation?

Commented [GU34]: is this how we call it throughout? or is this standard terminology?

Commented [nj35R34]: Standard terminology. I will add a section before "Graph generation" that explains all the graph basics the reader needs to know.

Commented [nj36]: This is what we currently use for our figures with edge weights. It would also be possible to show a figure for this comparing the main results between this minimum and maximum.

probability of learning both B and C. This means that in the minimal edge set, skipped foundational skills do not affect downstream learning, while in the maximal edge set, such gaps directly impact all dependent skills. See Fig. S12.

We therefore describe dependency constraints along three dimensions: prerequisite density (R), measuring the average number of prerequisites per trait; constraint mediation, ranging from minimal edge set (transitive reduction) to maximal edge set (transitive closure); and dependency strength, determining how strictly prerequisites must be satisfied.

2.5 Perceived Learnability

In our baseline model, trait interdependencies are fully opaque. We relax this assumption by allowing individuals to have knowledge about traits being “out of reach” for them (Fig. 2J-L). We model this with as a trait’s “perceived learnability” (L_i) using a transparency parameter θ :

$$L_i = (1 - \frac{M_i}{\sum_k M_k})^\theta$$

The transparency parameter controls the strength with which individuals discriminate between traits with different numbers of missing prerequisites M_i . As θ approaches 0, all traits are perceived to be learnable. As θ approaches infinity, only traits with no missing prerequisites are perceived to be learnable.

The probability that a trait is attempted for learning is the product of the normalized weight placed on the trait by the social learning strategy and the perceived learnability of the trait:

$$P_i = \frac{W_i}{\sum_k W_k} \cdot L_i$$

2.6 Markov Chain

We analyze our social learning model using two approaches. For networks with 8 or fewer traits, we use Markov chain analysis to compute transition probabilities between all possible repertoire states. For larger networks where the state space becomes computationally prohibitive, we use agent-based simulations that approximate the Markov chain dynamics (see 2.7). Additionally, we use a reinforcement learning agent-based model to examine how individuals adaptively adjust their social learning strategies based on experience with different constraint environments (see 2.8).

2.6.1 Constructing the Transition Matrix. The learner starts with a repertoire in which only the root trait is known. We use the adjacency matrix of the graph to convert the attempt probabilities to transition probabilities. First, attempt probabilities are normalized to a range of 0 to 1. Next, each attempt probability is multiplied by the probability that the trait is learnable, which is calculated as:

$$P(\text{trait } i \text{ is learnable}) = \prod_{j \in M_i} (1 - r_{ji})$$

Where M_i is the set of prerequisites for trait i that are not in the learner’s repertoire and r_{ji} is the weight of the edge from prerequisite trait j to focal trait i . The stronger the dependency relationship with a missing prerequisite (represented by its weight), the more significantly it decreases the probability of learning the trait, up to 0 if $r_{ji} = 1$ for any unknown trait. The probability that the learner will remain in the same state is equal to one minus the total of

Commented [NJ37]: I reworked this section, is it more clear now?

I am undecided whether I like this “mediation” framing, but not sure if I can come up with something better.

Commented [nj38]: R and edge set are not independent from each other which makes this 3 dimension model confusing and the visualization will open up more questions than it answers. Should we remove this whole paragraph?

Commented [LM39R38]: Yes I would remove this paragraph and substantially shorten the two paragraphs above. Can we just say:

When interdependencies are gradual, we cannot apply transitivity reduction in a straightforward way, we analyse the set of graphs we have in the previous analyses and reduce the edge weights. By means of a robustness test, we take the maximally constrained network ($R=3$ for the 8-trait case), and compare performance of social learning strategies in two ways: (1) with just reducing the requirements, and (2) with adding edges between all traits and their ancestors. Fig. S12 shows results between these extremes

Commented [nj40]: I want to remove the term mediation completely, it’s too confusing.

Commented [LM41R40]: Yes so i think this para can be dropped

Commented [LM42]: So moving the general formulation here would be nice

Commented [NJ43]: Add rationale for this formulation

Commented [LM44]: This is confusing as W was used for trait weights

Commented [nj45R44]: Changed it to r now, or would you prefer e to avoid mixing it up with R?

these learnability probabilities. By applying this calculation to each possible state within the constraints of the structure, we construct a transition matrix.

2.6.2 Computing outcome measures. To measure performance we track the expected payoff at each time step using standard Markov chain analysis (Otto & Day, 2007). We begin with an initial state distribution π_0 with 100% probability on the naive state. For each successive step, the distribution is updated by multiplying it with the transition matrix. The expected payoff at each step is computed as the dot product of the current state distribution and the vector of state payoffs. This calculation effectively weighs each state's payoff by the probability of being in that state at that particular time step.

Our results in Figures 1, 2 and 4 are presented showing the expected payoff at step

$t = \lfloor \frac{5}{8} \cdot n \rfloor$. To allow comparison of networks with varying n , we further divide expected payoff by step t . Because the average payoff of all traits is 1, a performance of 1 therefore indicates that the strategy achieves average performance over t steps. Figure S2 presents the expected payoffs for steps 1-10.

To measure the expected time until all traits in the structure are known (absorption), we first identify the submatrix Q of the transition matrix, which contains only states in which not all traits are known. The fundamental matrix $N = (I - Q)^{-1}$ is then computed by subtracting Q from the identity matrix I and inverting the result using LU decomposition with partial pivoting (Golub & Van Loan, 2013). Each entry N_{ij} in the fundamental matrix gives the expected number of steps spent in transient state j when starting from transient state i . The expected time to absorption from the initial state is the sum of the corresponding row in the fundamental matrix.

2.6.3 Frequencies of Traits and Repertoires. The Markov chain requires a distribution of repertoire frequencies in the background population as input, which can be used to compute corresponding trait frequencies. In the baseline model we assume that individuals in the background population express all traits in their repertoire with uniform probability (for robustness tests, see Fig. 2D,E). Trait frequencies in the background population therefore only depend on the prerequisite structure, such that traits with fewer prerequisites have higher frequency. Since the Markov chain requires trait frequencies as input, and we assume that trait frequencies should depend on how difficult each trait is to learn, we use a two-pass approximation. First, we construct a preliminary transition matrix assuming uniform trait frequencies and calculate the resulting quasi-stationary distribution using a random learning strategy. We then use the trait frequencies from this distribution as input for the final analysis. While this does not guarantee convergence to the true equilibrium frequencies, it provides a reasonable approximation for our purposes. The quasi-stationary distribution excludes the absorbing state in which all traits are known. We therefore assume that 5% of the population is always in the absorbing state, i.e., has learned all traits.

2.7 Agent-Based Approximation of the Markov Chain

For graphs with more than 8 traits the state space grows exponentially, increasing the computational demands on constructing the transition matrix. This burden arises because the transition matrix represents all possible state-to-state transitions, requiring entries for every potential repertoire state. To obtain the results for Fig. 2 B-C and Fig. 3 of the main text, we therefore approximate the results of the Markov chain using an agent-based model. Given a graph, we use a Monte Carlo simulation to initialize a background population of 10,000 naive learners who attempt to learn a random trait at each time step, using asocial (individual) learning. To control this process, we use a parameter determining the probability

Commented [LM46]: Probably good to signal in the first sentence already that we follow standard Markov chain procedures (take the reference for Otto & Day), and indicate where we deviate from standard methods. I guess Ingmar would appreciate it if we cite his book on Markov models?

Commented [nj47R46]: I think citing ingmar would look like unnecessary self-citation. His book is about hidden markov models so there are just better primary sources for markov chain basics

Commented [LM48]: Reference needed?

Commented [nj49R48]: Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.

Commented [NJ50]: This needs to be simpler. The important part is that this is a two-pass approach where we hope that the output trait frequencies are stable with respect to input trait frequencies.

Commented [nj51R50]: We don't have a good proof of that though. I did an analysis that shows that for most structures the output frequencies will become stable with respect to the input, but this would require repeated iteration, which we aren't doing.

Commented [LM52R50]: Here a reference would help. To console the reader we might show Fig. 1 for a few very different background distributions and refer to a supp figure with max 3 panels

Commented [nj53R50]: What do you think of the new text? Need to think about the figure you suggested. Not sure if there is a reference that would make sense for this.

Commented [NJ54]: This is basically just a monte carlo simulation

that an agent is replaced by a naive agent at each time step. This “reset rate” is adjusted to achieve a population in which 5% of agents have learned all traits, **mirroring the Markov chain frequencies**.

After estimating background trait frequencies, we initialize a population of 1,000 noninteracting agents with a single fixed social learning strategy. At each time step, the model calculates the weights placed on unlearned traits for each agent, using the social learning strategy and the frequencies of the background population, as described for the Markov chain. Finally, we calculate performance per time step as the average of state payoffs across agents at time t . Comparing results between the agent-based approximation and the Markov chain model, we find close agreement between the two approaches, with only **negligible differences** (see Fig S11).

Commented [NJ55]: Arguably this should also be verified with figures, but that's also a little cumbersome.

Commented [LM56R55]: Maybe for a (small) subset of cases, e.g., those shown in fig 1?

Commented [LM57R55]: So if linked to Fig. 1 or 2A it makes sense to have 1 robustness test for 8 traits

Commented [nj58R55]: @Lucas Molleman Seeing how close the agent-based model and the Markov chain are in their final results makes me think that verifying this is a bit unnecessary.

Commented [NJ59]: quantify

2.8 Agent-Based Model of Social Learning Strategy Preference

For the results shown in Fig. 4, we **simulate** a model in which agents are allowed to adjust their learning strategy based on experience. We initialize a population of 1000 agents (as described in 2.7) learning in 100 “learning domains”, each represented by a graphs. These graphs can either be fully constrained, (cf. Fig. 1, right hand side), or fully unconstrained (Fig. 1, left-hand side). To characterize the impact of experienced constraints on usage of social learning strategies, we varied the proportion p of fully constrained graphs.

Agent repertoires in each of the 100 domains are initialized using a population of random asocial learners as described in section 2.7. Each agent maintains Q-values $Q(s)$ for each social learning strategy s , representing the estimated expected payoff from using that strategy. All Q-values are initialized to 1.0, corresponding to the average trait payoff.

At each time step, a random agent and learning domain are sampled. The agent selects a strategy using softmax selection over all strategies with temperature $\tau = 0.05$:

$$P(s) = \frac{e^{Q(s)/\tau}}{\sum e^{Q/\tau}}$$

The agents samples $n = 30$ **demonstrators** and observes the traits they express for the current domain. The agent then selects a trait for learning according to their chosen social learning strategy and receives reward r (the trait's payoff if successful, 0 otherwise). The Q-value is then updated using reinforcement learning with learning rate $\alpha = 0.05$:

$$Q(s) \leftarrow Q(s) + \alpha(r - Q(s))$$

This allows agents to adapt their social learning strategies based on the success of previous learning attempts.

2.9 Sensitivity tests

To test the robustness of our main findings, we systematically varied key model assumptions while holding other parameters constant. All sensitivity tests use 8-trait networks unless otherwise specified.

2.9.1 Trait expression patterns. The baseline model assumes individuals express all traits in their repertoire with equal probability. We tested scenarios where individuals preferentially express certain types of traits by modifying the perceived trait frequencies used in social learning calculations (Fig. 2D-E). The observable frequencies f_p are biased according to:

Commented [LM60]: I guess the whole methods needs to be in past tense

Commented [nj61R60]: Wouldn't it make more sense to keep it in present? Or does science not like that.

Commented [nj62]: Show figure with 10

$$f_p = f \cdot bias^2$$

where f is the baseline trait frequency and the bias term varies by condition. For preferential expression of more advanced traits (Fig. 2D), the bias equals each trait's distance from the root node. For preferential expression of high-payoff traits (Fig. 2E), the bias equals each trait's payoff value. For strategies that depend on repertoire frequencies (prestige and proximal learning), we use iterative optimization with gradient descent to infer compatible state frequencies that would produce these biased trait frequencies.

2.9.2 Correlated payoffs and network structure. The baseline model assigns trait payoffs independently of network position (see 2.2). We tested scenarios where deeper traits systematically receive higher payoffs (Fig. 2F). Specifically, we sort traits by their distance from the root node and assign payoffs in ascending order, so that traits requiring more prerequisites receive higher rewards. This creates systematic correlation between trait advancement and reward value, causing payoff-biased strategies to focus on traits they cannot yet learn.

2.9.3 Skewed payoff distributions. Instead of the uniform payoff spacing described in 2.2, we tested highly skewed distributions where one trait receives a high payoff (2.0) and all others receive low payoffs (0.2), maintaining the same mean payoff of 1.0 (Fig. S7). This tests strategy performance when rewards are concentrated in a single trait.

2.9.4 Probabilistic dependencies. The baseline model uses deterministic prerequisite relationships (edge weights = 1.0). We tested probabilistic ("softened") dependencies by uniformly setting all edge weights to lower values (0.8, 0.5, and 0.2 in Fig. 2G-I). This represents scenarios where prerequisites are sometimes optional, allowing learners to occasionally bypass foundational requirements.

For probabilistic dependency tests, we used the transitive closure of each network structure, making all ancestor-descendant relationships explicit as direct edges (Fig. 2G-I). This represents a domain without prerequisite mediation, where the learnability of a trait depends on all its ancestors, making skipping to advanced traits more difficult. Results using the original transitive reduction (complete prerequisite mediation, skipping is easier) show qualitatively similar patterns but with smaller performance differences due to reduced overall constraints (Fig. S9).

2.9.5 Perceived learnability. The baseline model assumes complete opacity regarding trait interdependencies (see 2.5). We tested scenarios where learners have partial insight into network structure using transparency values 1, 10, 100 (Fig. 2J-L). Higher transparency values increasingly penalize traits with many missing prerequisites, representing greater structural knowledge that allows learners to focus on traits closer to their current skill level.

2.10 Inference of real-world interdependency structures

For the graphs for hunter-gatherer skills, we...

The interdependency structures for restaurant cooking and mathematics are derived following the approach of Hosseinoun et al. (2025). The authors use the O*NET 24.4 (2019) database, which contains estimates of required proficiency on 120 skills for almost 1,000 occupations in the US economy to estimate a dependency structure among skills. In our analysis, we focus on two occupations, restaurant cooks and mathematical technicians. To

Commented [nj63]: This is a little silly, we should apply the expression bias directly in the proximal and prestige implementations rather than adjusting state frequencies

Commented [NJ64]: Add ref to SI figure with transparency and soft edges

obtain interdependency structures for an occupation, we subset the full 120 skill graph to only include skills that are required for an occupation and their ancestors. An interactive visualization of the full interdependency structure and subsetting is available at https://nanda2502.shinyapps.io/occ_graph/